



Debugging

Module 6

Learnings covered in this Unit



Debugging Introduction



Debugging Basics



Debugging PowerShell Console



Debugging PowerShell ISE



Debugging Advanced



Debugging Other commands

Debugging Introduction

Debugging Overview

What is debugging?

- Process that allows you to examine your code while it's running

How does it help?

- Enables you to identify errors and correct them during execution

To which code does it apply?

- Debugging works with commands, expressions, functions, workflows and scripts

Debugging Features

Available in console and ISE (starting PowerShell 5 also support for remote debugging in ISE)

\$PSDebugContext to determine if we are debugging

\$DebugPreference influences debugging behavior

-Debug common parameter to influence \$DebugPreference

Cmdlets include:

- Breakpoint
- Call stack
- Debug
- Trace
- StrictMode

Breakpoints – automatic variables

Determine whether we are debugging or not (e.g. from an automation perspective)

When debugging commences the current scope doesn't change

Example: if debugging starts within a function it remains in the function

```
PS C:\> if($PSDebugContext) {  
    Write-Output "Currently debugging"  
} else {  
    Write-Output "Not debugging"  
}
```

Breakpoints – automatic variables

```
PS C:\> Function Bits {  
>> Get-Service -Name Bits  
>> }
```

```
PS C:\> Set-PSBreakpoint -Command Get-Service
```

```
PS C:\> Bits  
Entering debug mode. Use h or ? for help.
```

```
PS C:\>> $PSDebugContext
```

```
InvocationInfo  
-----
```

```
System.Management.Automation.InvocationInfo {Command breakpoint on  
'Get-Service'}
```

```
Breakpoints  
-----
```

\$DebugPreference

Automatic Variable

Debug information is not displayed by default

To display debug information:

1. Change **\$DebugPreference**
2. Use **-Debug** common parameter

Valid Values

SilentlyContinue
(Default)

- Debug message is not displayed, and execution continues without interruption

Stop

- Displays debug message and stops execution
- Writes error to console

Inquire

- Displays debug message and prompts user to continue

Continue

- Displays debug message and continues with execution

Debug common parameter

```
PS C:\> Function Bits {  
>> [Cmdletbinding()]Param()  
>> $Service = Get-Service -Name Bits  
>> $Service  
>> Write-Debug "Debug enabled: status is $($Service.Status)"  
>> }
```

```
PS C:\> Bits
```

Status	Name	DisplayName
-----	----	-----
Running	Bits	Background Intelligent Transfer Ser...

Debug common parameter

```
PS C:\> Bits -Debug
```

Status	Name	DisplayName
-----	----	-----
Running	Bits	Background Intelligent Transfer Service...

Custom Debug
Message

DEBUG: Debug enabled: the status of service BITS is Running

Confirm

Continue with this operation?

[Y] Yes [A] Yes to All [H] Halt Command [S] Suspend [?] Help (default is "Y"):

Demonstration

Debugging introduction



Questions?



Debugging Basics

Breakpoints

- A designated spot in a script where you pause the operation to examine the current state of the variables and the environment in which your script is running

From [How to Debug Scripts in Windows PowerShell ISE](#)

- The prompt will change and is now prefixed with "[DBG]:"

Breakpoints

While in the debugger environment you can:



Examine a
variable



Query the
runtime state



Test conditions

You can set breakpoints at:



Variables (Read /
Write / ReadWrite)



Commands (cmdlet,
function or workflow)



Line and column
numbers

Breakpoints

Breakpoints for variables have the limitations:

- Does not work with \$_
- Does not work with \$input
- Does not work with \$Args
- Does not work with \$PSBoundParameters
- Does not work with \$MyInvocation

How to set Breakpoints

The Windows PowerShell ISE includes menu items that map to some of these cmdlets.

Description	Command
Gets breakpoints in current session	<code>Get-PSBreakpoint</code>
Sets breakpoints on lines, variables, or commands	<code>Set-PSBreakpoint</code>
Turns off breakpoints in current session	<code>Disable-PSBreakpoint</code>
Re-enables breakpoints in current session	<code>Enable-PSBreakpoint</code>
Deletes breakpoints from current session	<code>Remove-PSBreakpoint</code>

Demonstration

Debugging basics



Questions?



Debugging PowerShell ISE

ISE Breakpoints

Set a breakpoint (scripts must be saved):

- Debug menu
- Right-click line Toggle breakpoint
- Press F9

Cut	Ctrl+X
Copy	Ctrl+C
Paste	Ctrl+V
Start Intellisense	Ctrl+Space
Start Snippets	Ctrl+J
Run Selection	F8
Toggle Breakpoint	F9

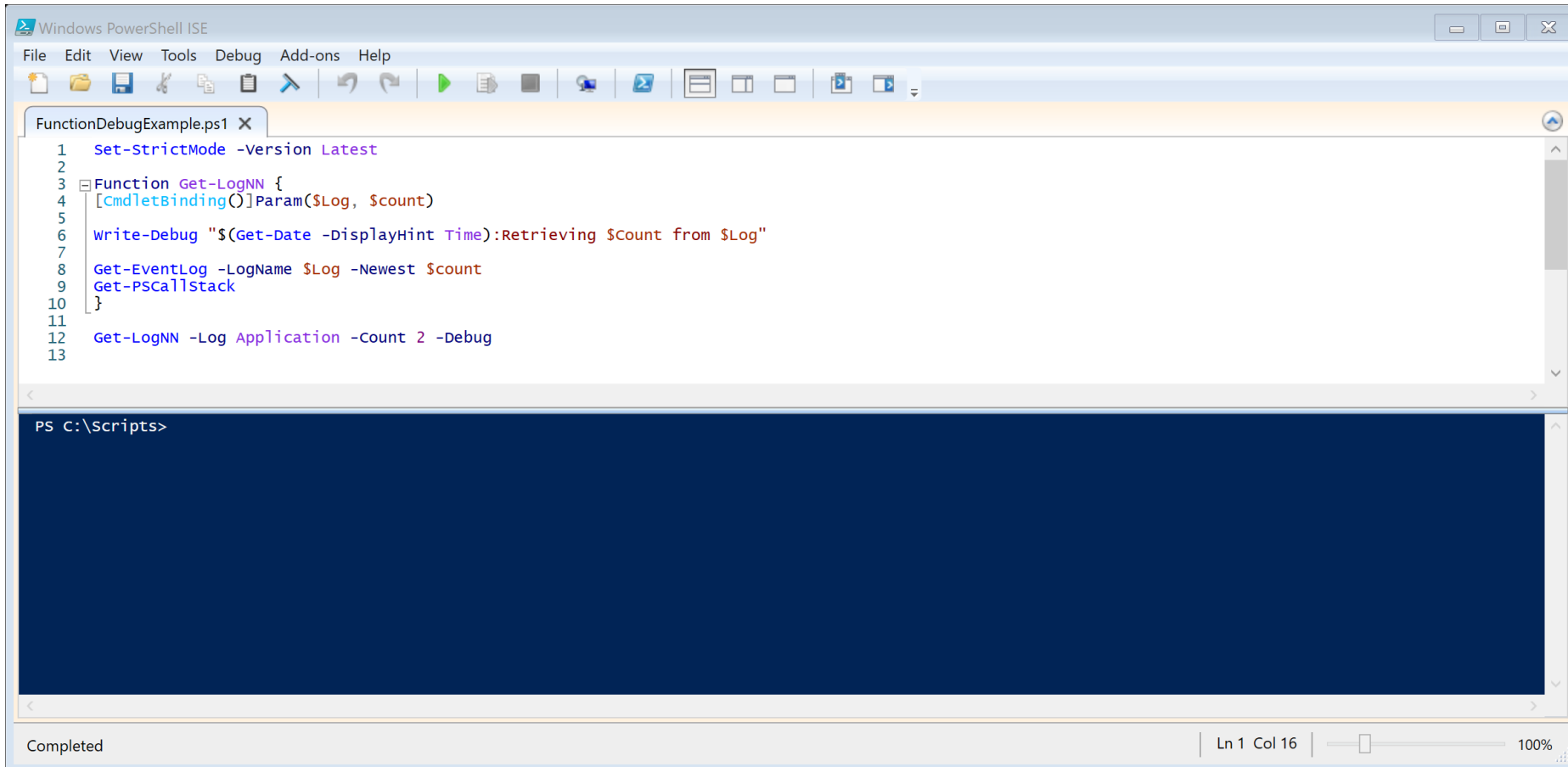
List/Remove/Enable/Disable breakpoints:

- Debug menu
- Ctrl+Shift+L/Ctrl+Shift+F9

File	Edit	View	Tools	Debug	Add-ons	Help
				Step Over	F10	
				Step Into	F11	
				Step Out	Shift+F11	
				Run/Continue	F5	
				Stop Debugger	Shift+F5	
				Toggle Breakpoint	F9	
				Remove All Breakpoints	Ctrl+Shift+F9	
				Enable All Breakpoints		
				Disable All Breakpoints		
				List Breakpoints	Ctrl+Shift+L	
				Display Call Stack	Ctrl+Shift+D	

ISE Breakpoints

Consider the following script (must be saved before a breakpoint can be set)



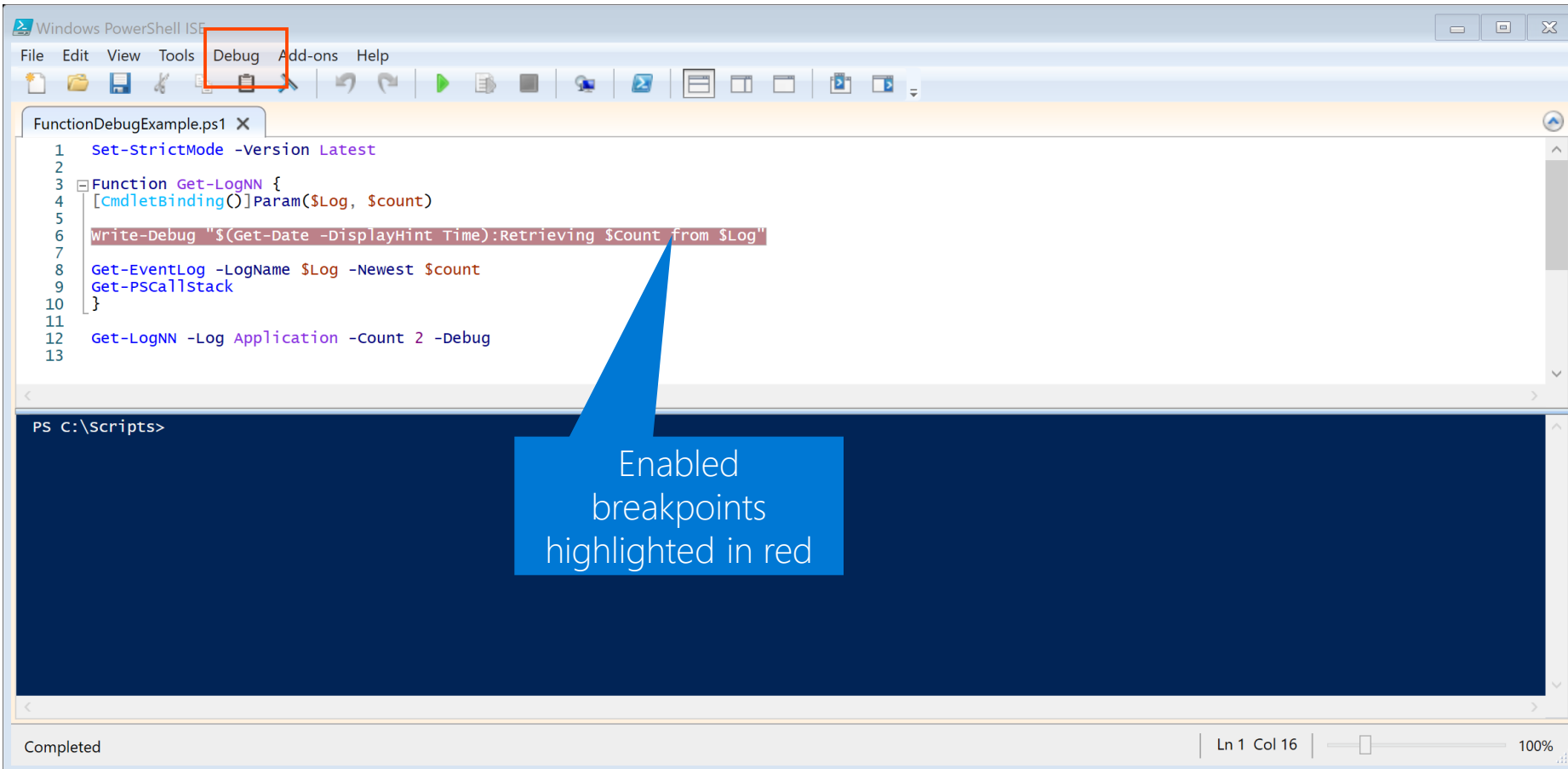
The screenshot shows the Windows PowerShell ISE interface. The menu bar includes File, Edit, View, Tools, Debug, Add-ons, and Help. The toolbar contains icons for file operations, execution, and debugging. The script editor displays a file named 'FunctionDebugExample.ps1' with the following code:

```
1 Set-StrictMode -Version Latest
2
3 Function Get-LogNN {
4     [CmdletBinding()] Param($Log, $count)
5
6     Write-Debug "$(Get-Date -DisplayHint Time):Retrieving $count from $Log"
7
8     Get-EventLog -LogName $Log -Newest $count
9     Get-PSCallstack
10 }
11
12 Get-LogNN -Log Application -Count 2 -Debug
13
```

A breakpoint is set on line 4, indicated by a blue dot in the left margin. The console window at the bottom shows the prompt 'PS C:\Scripts>' and the status bar at the bottom indicates 'Completed', 'Ln 1 Col 16', and '100%' zoom.

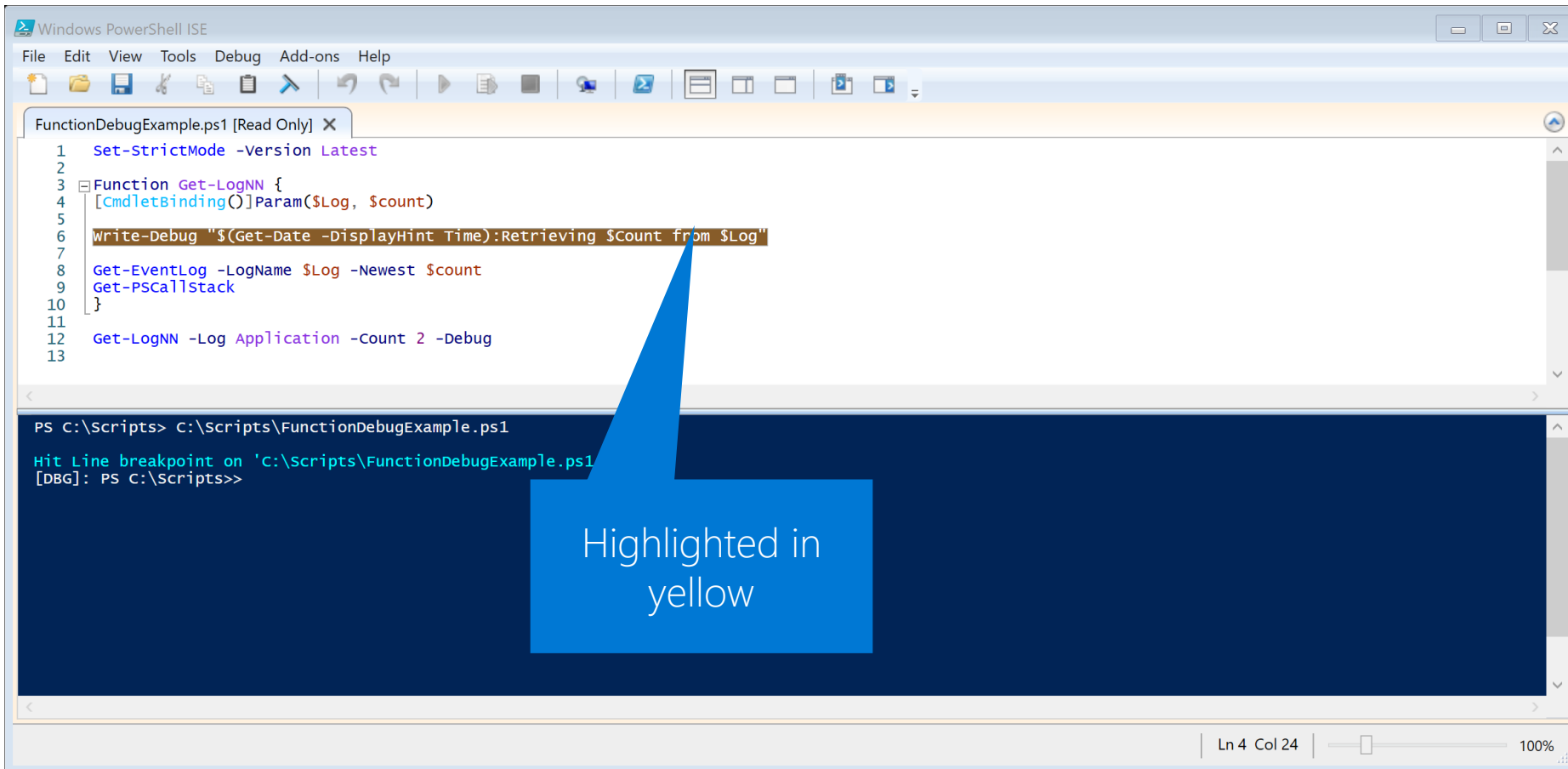
ISE Breakpoints

Breakpoint set using F9 or the Toggle Breakpoint from the Debug menu



ISE Breakpoints

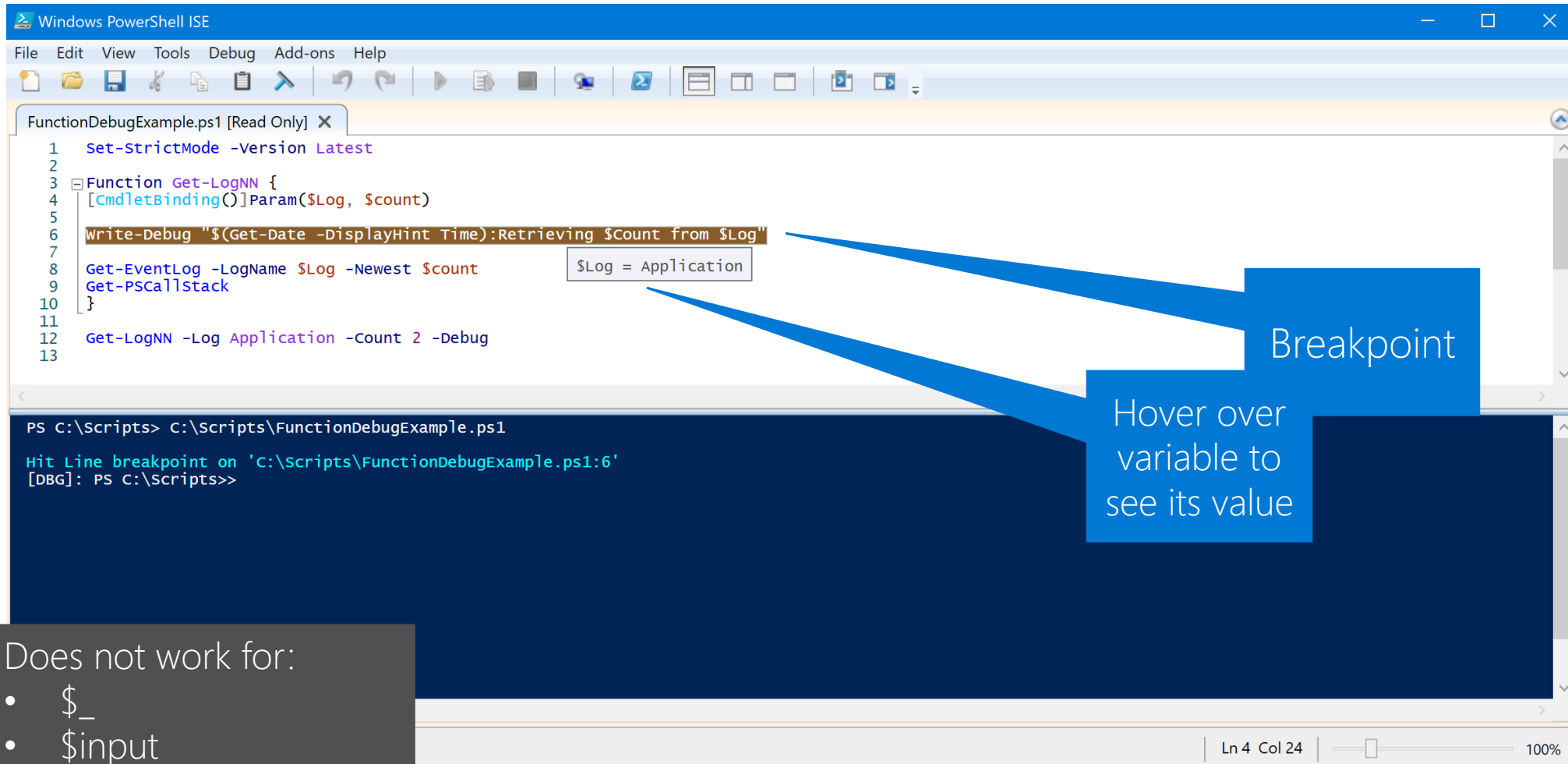
Breakpoint is highlighted in yellow when reached during execution



ISE Debugger Commands

Debugging task	Description	Shortcut
Step Into	Runs current statement and stops at next one; Steps into function or script if next statement is function or script call	F11
Step Over	Runs current statement and stops at next one; Executes function or script if next statement is function or script call but doesn't step into	F10
Step Out	Steps out of current function and goes up one level if current function is nested; If in main body executes until end or next breakpoint	SHIFT+F11
Continue	Continues to the end or next breakpoint Function and invocations are executed but not stepped through	F5
Stop	Stops the debugger and resumes normal operation	SHIFT+F5

ISE Breakpoints – Variables



Does not work for:

- \$_
- \$input
- \$PSBoundParameters
- \$Args
- \$MyInvocation

Demonstration

Debugging with PowerShell ISE



Questions?



Debugging PowerShell Console

Command-Line Debugging

- Full debugging experience in the PowerShell console
- Enables you to debug a script at runtime when the ISE is not available for use

```
PS C:\> Set-PSBreakPoint -Script .\ScriptHelpExample.ps1 -Variable log -Mode Read
```

ID	Script	Line	Command	Variable	Action
--	-----	----	-----	-----	-----
7	ScriptHelpExample.ps1			log	

```
PS C:\> .\ScriptHelpExample.ps1 -log system -numberofevents 2
```

```
Hit Variable breakpoint on 'C:\ScriptHelpExample.ps1:$log' (Read access)
```

```
[DBG]: PS C:\>>
```

Debugger Commands

```
[DBG]: PS C:\Scripts>> ?
```

s, stepInto	Single step (step into functions, scripts, etc.)
v, stepOver etc.)	Step to next statement (step over functions, scripts, etc.)
o, stepOut	Step out of the current function, script, etc.
c, continue	Continue operation
q, quit	Stop operation and exit the debugger
k, Get-PSCallStack	Display call stack
l, list	List source code for the current script. Use "list" to start from the current line, "list <m>" to start from line <m>, and "list <m> <n>" to list <n> lines starting from line <m>
<enter>	Repeat last command if it was stepInto, stepOver or list
?, h	displays this help message.

Debugging Functions

```
function Test-DebugFunction {  
    begin {  
        Write-Host "Begin"  
    }  
    process {  
        Write-Host "Process"  
    }  
    end {  
        Write-Host "End"  
    }  
}
```

```
PS C:\> Set-PSBreakpoint -Command Test-  
DebugFunction  
PS C:\> Test-DebugFunction  
Hit Command breakpoint on 'Test-DebugFunction'  
Stopped at: begin {  
[DBG]: PS C:\>>
```

Begin, Process, and End
blocks

Break at each section

Demonstration

Debugging with PowerShell
Console

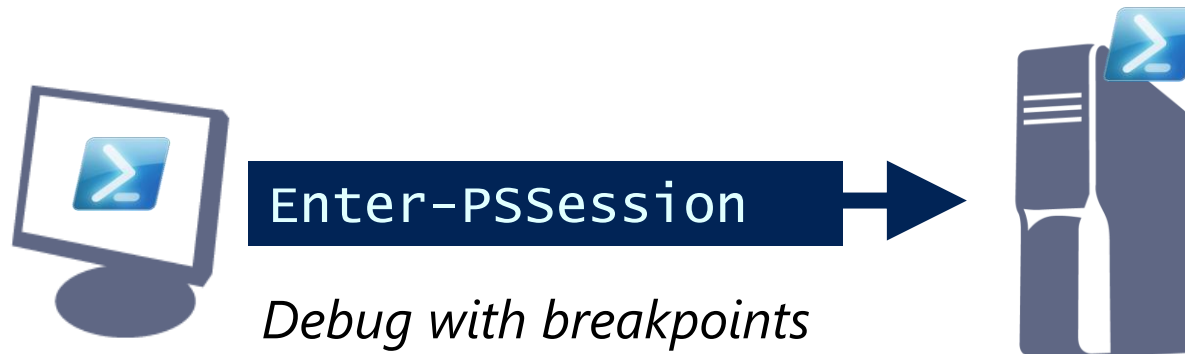


Questions?



Debugging Advanced

Debugging Remote Scripts



If script hits breakpoint, client starts debugger

If disconnected session is at breakpoint, Enter-PSSession starts debugger when reconnecting

Debugging remote scripts

- Debugging remote scripts is only possible from the PowerShell console
starting PowerShell 5 also support for remote debugging in ISE
- Remote debugging will prefix the prompt with "[REMOTE_HOSTNAME]:
[DBG]:"

Typical steps to initiate remote debugging:

- **Enter-PSSession** to connect to remote host
- **Set-PSBreakPoint -Script <script_to_debug> -Line 1**
- Allows exit the remote host (e.g. Exit-PSSession) to make sure you don't leave any remote sessions open

Example: Debugging remote script

```
PS C:\> Enter-PSSession -ComputerName DC
```

```
[DC]: PS C:\> Set-PSBreakpoint -Script .\ScriptExample.ps1 -Line 1
```

ID	Script	Line	Command	Variable	Action
0	ScriptExample.ps1	1			

```
[DC]: PS C:\> .\ScriptExample.ps1
```

```
Entering debug mode. Use h or ? for help.
```

```
Hit Line breakpoint on 'C:\ScriptExample.ps1:1'
```

```
At C:\ScriptEx.ps1:14 char:5
```

```
+ Get-EventLog -LogName $Log -Newest $Count
```

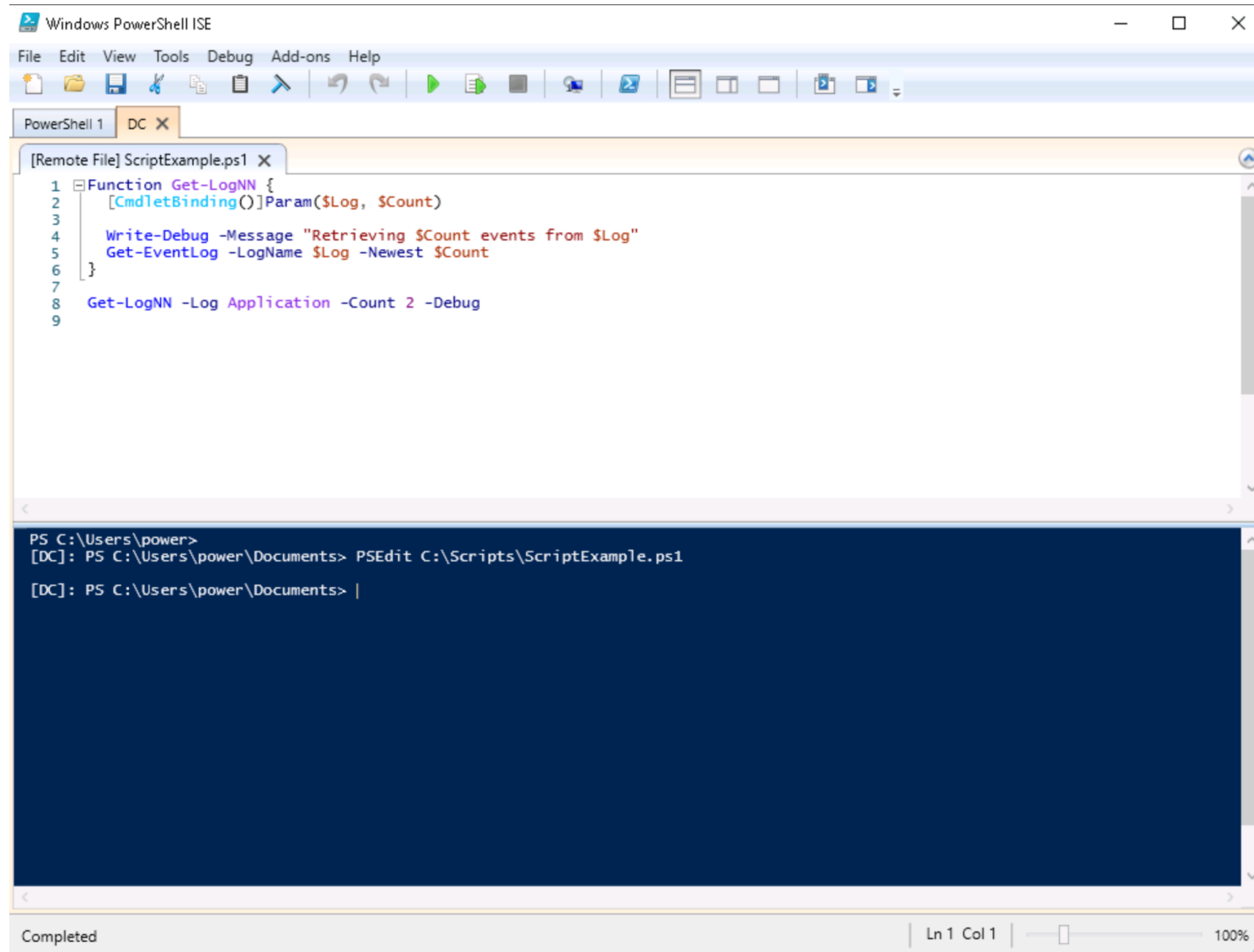
```
[DC]: [DBG]: PS C:\>>
```

Debugging remote scripts with ISE

Typical steps to initiate remote debugging in ISE:

- Start the PowerShell ISE
- Select File → New Remote PowerShell Tab (shortcut use CTRL+Shift+R)
- Provide remote computer and username to connect
- Provide password to complete the connection
- **PSEdit <script file>** to open script file
- Use **F9** to toggle breakpoints
- Run the script to start debugging

Debugging remote scripts with ISE



The screenshot shows the Windows PowerShell ISE interface. The top menu bar includes File, Edit, View, Tools, Debug, Add-ons, and Help. Below the menu is a toolbar with various icons. The main editor area displays a script file named 'ScriptExample.ps1' with the following content:

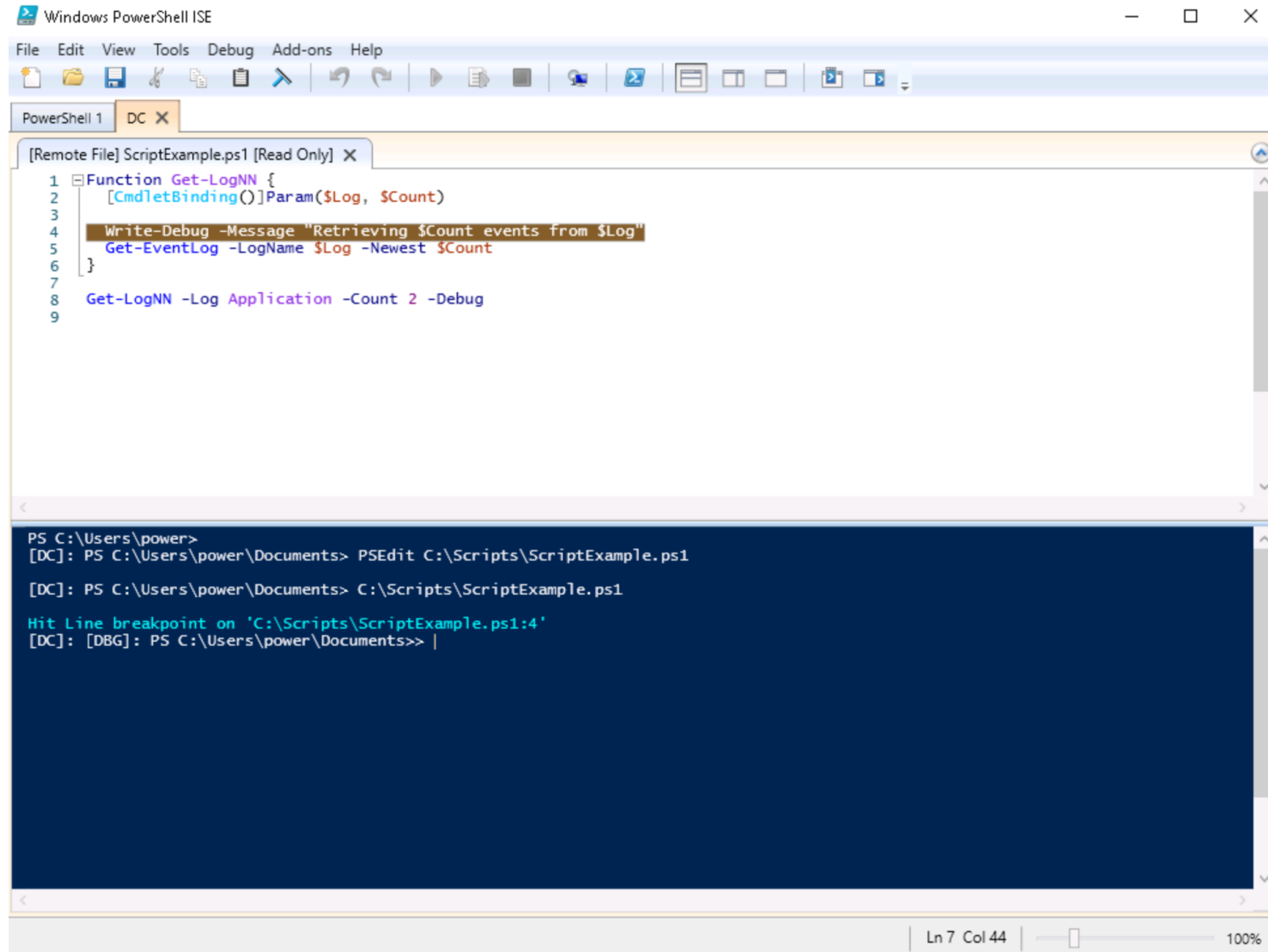
```
1 Function Get-LogNN {  
2     [CmdletBinding()] Param($Log, $Count)  
3  
4     Write-Debug -Message "Retrieving $Count events from $Log"  
5     Get-EventLog -LogName $Log -Newest $Count  
6 }  
7  
8 Get-LogNN -Log Application -Count 2 -Debug  
9
```

Below the script editor is a console window with the following text:

```
PS C:\Users\power>  
[DC]: PS C:\Users\power\Documents> PSEdit C:\Scripts\ScriptExample.ps1  
[DC]: PS C:\Users\power\Documents> |
```

The status bar at the bottom indicates 'Completed' and shows the cursor position as 'Ln 1 Col 1' with a zoom level of '100%'.

Debugging remote scripts with ISE



The screenshot shows the Windows PowerShell ISE interface. The top menu bar includes File, Edit, View, Tools, Debug, Add-ons, and Help. Below the menu is a toolbar with various icons. The main editor area displays a script titled "[Remote File] ScriptExample.ps1 [Read Only]". The script content is as follows:

```
1 Function Get-LogNN {  
2     [CmdletBinding()] Param($Log, $Count)  
3  
4     Write-Debug -Message "Retrieving $Count events from $Log"  
5     Get-EventLog -LogName $Log -Newest $Count  
6 }  
7  
8 Get-LogNN -Log Application -Count 2 -Debug  
9
```

The bottom console area shows the following output:

```
PS C:\Users\power>  
[DC]: PS C:\Users\power\Documents> PSEdit C:\Scripts\ScriptExample.ps1  
  
[DC]: PS C:\Users\power\Documents> C:\Scripts\ScriptExample.ps1  
  
Hit Line breakpoint on 'C:\Scripts\ScriptExample.ps1:4'  
[DC]: [DBG]: PS C:\Users\power\Documents>> |
```

The status bar at the bottom indicates the current position is at Line 7, Column 44, and the zoom level is 100%.

Debugging and scopes

Scope: Area ("territory") that a given object is visible or available

PowerShell protects access to: variables, aliases, functions, and PSDrives by limiting the lifetime and visibility (or *scope*) where they can be read/changed

Created when
starting a
command shell,
script, or function

Cleaned when
script or function
finishes

Defines separate
items in memory
(e.g. variables)

Avoids naming
collisions of items
across different
scopes

Debugging and scopes

Four scopes available:

- Global (*PowerShell Host[session]*)
- Local (*Default*)
- Script
- Private

Debugging and scopes

- Whenever you enter the debug environment, you remain in the same scope
- The following example shows same scope although entered the debug environment

```
PS C:\> $DebugExample = "Test"
PS C:\> Set-PSBreakpoint -Script C:\Scripts\DebugExample.ps1 -Line 1
```

ID	Script	Line	Command	Variable	Action
--	-----	----	-----	-----	-----
4	DebugExample.ps1	1			

```
PS C:\> C:\Scripts\DebugExample.ps1

[DBG]: PS C:\> $DebugExample
Test
```

Demonstration

Debugging - advanced



Questions?



Debugging Other commands

Debugging scripts

Cmdlet	Description
Set-PSDebug -Off	Turns off all script debugging
Set-PSDebug -Step	Before each line is executed, the user is prompted to either stop, continue, or enter new interpreter level to inspect the state of script
Set-PSDebug -Strict	Throws exception if a variable is referenced before a value is assigned to it
Set-PSDebug -Trace <int32>	Specifies trace level: 0 = turns off tracing 1 = each line 2 = variable assignments, function calls, and script calls

Helps in debugging scripts.

Allows you to set the trace level, toggles strict mode, and toggles the use of steps

Deprecated and replaced by: Set-StrictMode

Note: affects the global scope

Example: Trace with Set-PSDebug

```
Set-PSDebug -Trace 1
function Get-LogNN {
    [CmdletBinding()]
    Param ($log, $count)
    Write-Debug "$(Get-Date -DisplayHint Time):Retrieving $count from $log"
    Get-EventLog -LogName $log -Newest $count
}
Get-LogNN -log system -count 2
```

```
PS C:\> .\fDebugEx.ps1
```

```
DEBUG: 7+ >>>> Get-LogNN -log system -count 2
```

```
DEBUG: 2+ function Get-LogNN >>>> {
```

```
DEBUG: 5+ >>>> Write-Debug "$(Get-Date -DisplayHint Time):Retrieving $count from $log"
```

```
DEBUG: 5+ Write-Debug "$( >>>> Get-Date -DisplayHint Time):Retrieving $count from $log"
```

```
DEBUG: 6+ >>>> Get-EventLog -LogName $log -Newest $count}
```

Index	Time	EntryType	Source	InstanceId	Message
-----	----	-----	-----	-----	-----
47851	Feb 25 13:48	Information	Micros...	1501	The Gro...
DEBUG:	6+	Get-EventLog	-LogName \$log	-Newest \$count	>>>> }

Set-StrictMode

Cmdlet Name	Description
Set-StrictMode -Version 1, 2 or Latest (3) Note: 'Latest' is strictest version	• Affects only current scope and its child scopes • Specifies conditions that cause an error in strict mode • Version determines level of enforced coding rules • Errors when code violates best-practice, i.e. ○ Reference uninitialized variable → Version 1, 2, Latest ○ Reference non-existent property → Version 2, Latest ○ Incorrectly called function e.g. put arguments in parentheses or, separate arguments using commas → Version 2, Latest ○ Variable w/o a name e.g. \${} → Version 2, Latest
Set-StrictMode -Off	• Turns strict mode off • Also turns off "Set-PSDebug -Strict"

Set-StrictMode - example

```
PS C:\> Get-Variable -Name E*
```

Name	Value
----	-----
Error	{}
ErrorActionPreference	Continue
ErrorView	NormalView
ExecutionContext	System.Management.Automation.Engine.Intrinsics



```
PS C:\> $Error
```

```
PS C:\> Set-StrictMode -Version Latest
```

```
PS C:\> $Error
```

```
The variable '$Error' cannot be retrieved because it has not been set.
```

```
At line:1 char:1
```

```
+ $Error
```

```
+ ~~~~~
```

```
+ CategoryInfo          : InvalidOperation: (ErrorString) [], RuntimeEx...
```

```
+ FullyQualifiedErrorId: VariableIsUndefined
```

Set-StrictMode - example

```
PS C:\> Get-ChildItem -Directory | Get-Member -MemberType CodeProperty
```

Name	MemberType	Definition
----	-----	-----
LinkType	CodeProperty	System.String LinkType{get=GetLinkType;}
Mode	CodeProperty	System.String Mode{get=Mode;}
Target	CodeProperty	System.Collections.Generic.IEnumerable`1[[System.S...

```
PS C:\> (Get-Item -Path $HOME).Modes
```

```
PS C:\> Set-StrictMode -Version Latest
```

```
PS C:\> (Get-Item -Path $HOME).Modes
```

The property 'Modes' cannot be found on this object. Verify that the prope...

At line:1 char:1

+ (Get-Item -Path \$HOME).Modes

+ ~~~~~

+ CategoryInfo : NotSpecified: (:) [], PropertyNotFoundException...

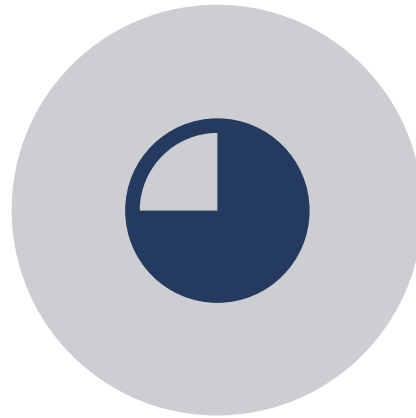
+ FullyQualifiedErrorId: PropertyNotFoundStrict

```
PS C:\>
```

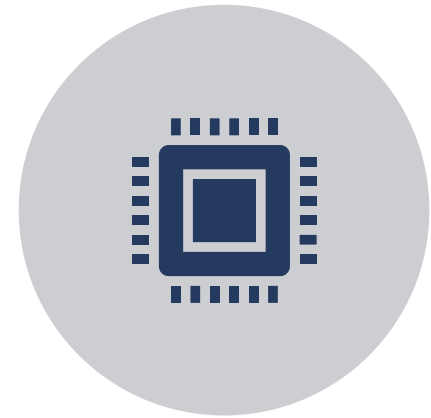
Get-PSCallStack



THE CALL STACK IS THE HISTORY OF
ALL SCRIPTS OR FUNCTIONS THAT
WERE RUN



ALLOWS DISPLAYING OF THE CALL
STACK OUTSIDE THE DEBUGGER FOR
ANY SCRIPT OR FUNCTION



ORIGINALLY DESIGNED TO BE USED
WITH THE DEBUGGER (SHORTCUT K
IN THE DEBUG ENVIRONMENT)

Get-PSCallStack

```
function Get-LogNN {  
    Param ($log, $count)  
    Get-EventLog -LogName $log -Newest $count  
    Get-PSCallStack  
}
```

```
PS C:\> Get-LogNN -log system -count 2  
ScriptName      :  
ScriptLineNumber : 4  
InvocationInfo  : System.Management.Automation.InvocationInfo  
Position       : Get-PSCallStack  
FunctionName    : Get-LogNN  
Command        : Get-LogNN  
Location       : <No file>  
Arguments      : {log=system, count=2}
```

Trace-Command

Parameter	Description
-PSHost	Indicates that the cmdlet sends the trace output to the PowerShell host. This parameter also selects the PSHost trace listener.
-Debugger	Debugger can be any user-mode or kernel-mode debugger or Visual Studio
-FilePath	Specifies a file that the cmdlet sends the trace output to. This parameter also selects the file trace listener.
-Option	- Type of events to trace - E.g. None, Constructor, All, Warning, Error, Scope
-Name	- Determines PowerShell components to be traced - Too many to list, and different for each machine, list with: Get-TraceSource Sort-Object Name - E.g. parameterbinding, typeconversion, cmdlet

- Allows us to configure and start a trace of any expression or command
- Displays detailed information about different types of events that are traced
- Traces all event types by default

Example: Trace parameter binding

```
PS C:\> Trace-Command -PSHost -Expression {Start-Process Notepad} `
-Option all -Name parameterbinding
DEBUG:1+>>>> Trace-Command -PSHost -Expression {Start-Process Notepad} -Option all -
Name parameterbinding
DEBUG:1+Trace-Command -PSHost -Expression >>>> {Start-Process Notepad} -Option all -
Name parameterbinding
DEBUG:ParameterBinding Information: 0 : BIND NAMED cmd line args [Start-Process]
DEBUG:ParameterBinding Information: 0 : BIND POSITIONAL cmd line args [Start-Process]
DEBUG:ParameterBinding Information: 0 : BIND arg [Notepad] to parameter [FilePath]
DEBUG:ParameterBinding Information: 0 : Executing VALIDATION metadata:
[System.Management.Automation.ValidateNotNullOrEmptyAttribute]
DEBUG:ParameterBinding Information: 0 : BIND arg [Notepad] to param [FilePath]
SUCCESSFUL
DEBUG:ParameterBinding Information: 0 : MANDATORY PARAMETER CHECK on cmdlet [Start-
Process]
DEBUG:ParameterBinding Information: 0 : CALLING BeginProcessing
DEBUG:ParameterBinding Information: 0 : CALLING EndProcessing
DEBUG:1+Trace-Command -PSHost -Expression {Start-Process Notepad >>>> } -Option all -
Name parameterbinding
```


Demonstration

Debugging - other commands



Questions?



Debugging

